

1. 如何设计有效的提示 2. 如何判断生成的响应是否合适 3. 如何设计保障措施和安全缓解策略，得到符合道德、符合伦理的回复

一. 基础提示工程 **Basic prompt engineering**

1. 提示的基本成分

指令本身和与指令相关的数据

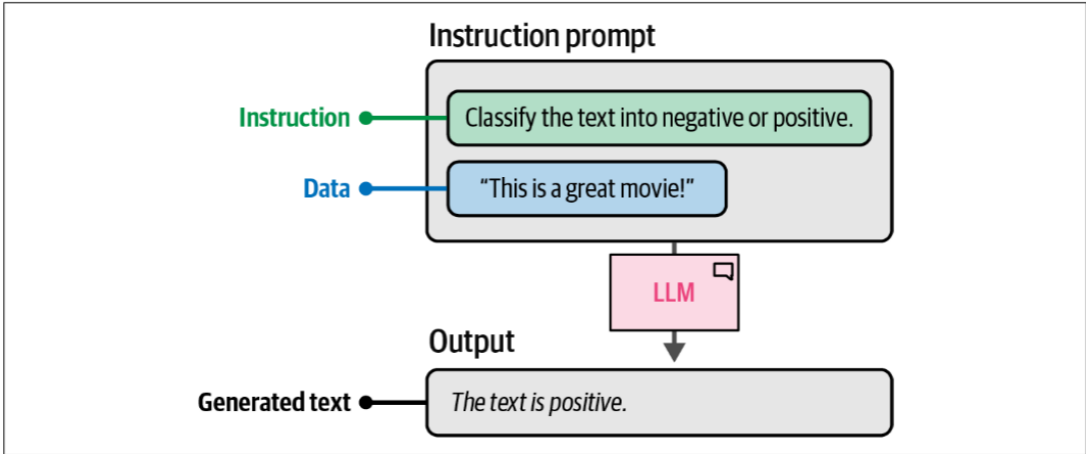


Figure 6-7. Two components of a basic instruction prompt: the instruction itself and the data it refers to.

Instruction-Based Prompting

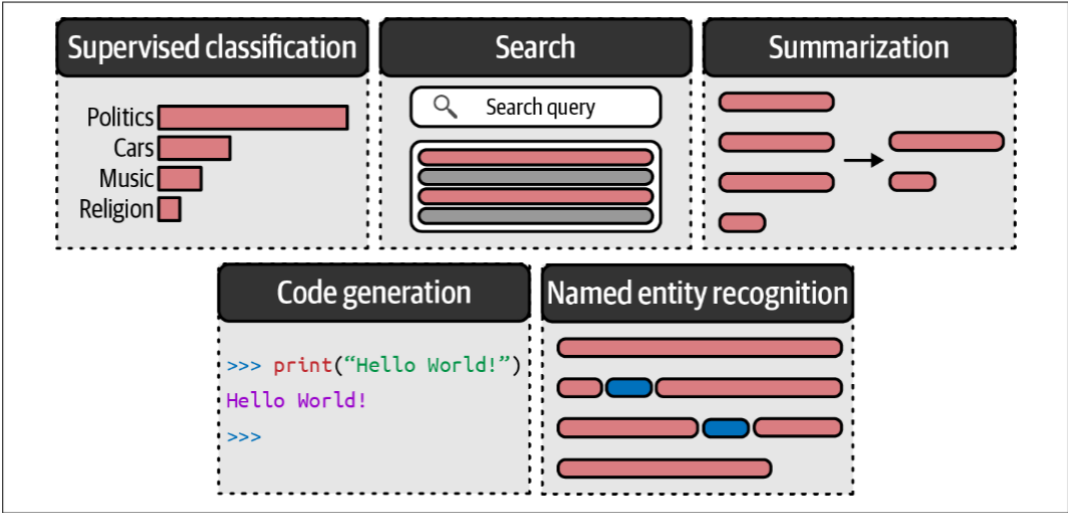


Figure 6-9. Use cases for instruction-based prompting.

为了确保模型只输出“负”或“正”，我们可以引入有助于指导模型的输出指标。在图 6-8 中，我们用“Text:”作为句子的前缀，并添加“Sentiment:”，以防止模型生成完整的句子。

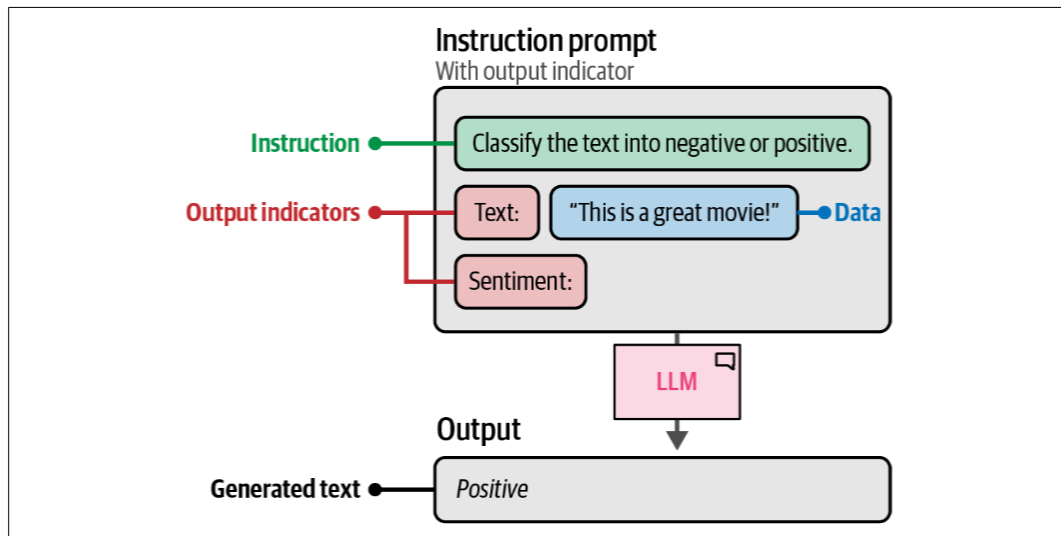


Figure 6-8. Extending the prompt with an output indicator that allows for a specific output.

Special techniques to get better responses:

1. Specificity: 准确描述你想要大模型去实现的任务是什么
2. Hallucination: 有时大模型会生成错误的回复，为了减少这种情况发生，我们可以让 LLM 只生成它知道的答案。如果它不知道，就让它回复“不知道”。
3. Order: 在你给出的提示的开头或结尾添加指令，特别是当提示较长时，因为 LLM 往往更关注提示开头的信息（primacy effect）和提示末尾的信息（recency effect）。

二. 高级提示工程 Advanced prompt engineering

1. Persona
2. Instruction
3. Context
4. Format
5. Audience
6. Tone
7. Data

Persona

Describe what role the LLM should take on. For example, use “You are an expert in astrophysics” if you want to ask a question about astrophysics.

Instruction

The task itself. Make sure this is as specific as possible. We do not want to leave much room for interpretation.

Context

Additional information describing the context of the problem or task. It answers questions like “What is the reason for the instruction?”

Format

The format the LLM should use to output the generated text. Without it, the LLM will come up with a format itself, which is troublesome in automated systems.

Audience

The target of the generated text. This also describes the level of the generated output. For education purposes, it is often helpful to use ELI5 (“Explain it like I’m 5”).

Tone

The tone of voice the LLM should use in the generated text. If you are writing a formal email to your boss, you might not want to use an informal tone of voice.

Data

The main data related to the task itself.

```
# Text to summarize which we stole from https://lalamor.github.io/illustrated-transformer/ :)
text = """In the previous post, we looked at Attention - a ubiquitous method in modern deep learning models. Attention is a concept that helped improve the performance of neural machine transla
The Transformer was proposed in the paper Attention is All You Need. A TensorFlow implementation of it is available as a part of the Tensor2Tensor package. Harvard's NLP group created a guide
Let's begin by looking at the model as a single black box. In a machine translation application, it would take a sentence in one language, and output its translation in another.
Popping open that Optimus Prime goodness, we see an encoding component, a decoding component, and connections between them.
The encoding component is a stack of encoders (the paper stacks six of them on top of each other - there's nothing magical about the number six, one can definitely experiment with other arra
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers:
The encoder's inputs first flow through a self-attention layer - a layer that helps the encoder look at other words in the input sentence as it encodes a specific word. We'll look closer at
The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is independently applied to each position.
The decoder has both these layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence (similar what attention does in seq2seq models).
Now that we've seen the major components of the model, let's start to look at the various vectors/tensors and how they flow between these components to turn the input of a trained model into
As to the case in NLP applications in general, we begin by turning each input word into a vector using an embedding algorithm.
Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.
The embedding only happens in the bottom-most encoder. The abstraction that is common to all the encoders is that they receive a list of vectors each of the size 512 - In the bottom encoder
After embedding the words in our input sequence, each of them flows through each of the two layers of the encoder.
Here we begin to see one key property of the Transformer, which is that the word in each position flows through its own path in the encoder. There are dependencies between these paths in the
Next, we'll switch up the example to a shorter sentence and we'll look at what happens in each sub-layer of the encoder.
Now we're Encoding
As we've mentioned already, an encoder receives a list of vectors as input. It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural netwo
"""
```

```
# Prompt components
persona = "You are an expert in Large Language models. You excel at breaking down complex papers into digestible summaries.\n"
instruction = "Summarize the key findings of the paper provided.\n"
context = "Your summary should extract the most crucial points that can help researchers quickly understand the most vital information of the paper.\n"
data_format = "Create a bullet-point summary that outlines the method. Follow this up with a concise paragraph that encapsulates the main results.\n"
audience = "The summary is designed for busy researchers that quickly need to grasp the newest trends in Large Language Models.\n"
tone = "The tone should be professional and clear.\n"
text = "MY TEXT TO SUMMARIZE" # Replace with your own text to summarize
data = f"Text to summarize: {text}"

# The full prompt - remove and add pieces to view its impact on the generated output
query = persona + instruction + context + data_format + audience + tone + data
```

```
messages = [
    {"role": "user", "content": query}
]
print(tokenizer.apply_chat_template(messages, tokenize=False))

<|user|>
You are an expert in Large Language models. You excel at breaking down complex papers into digestible summaries.
Summarize the key findings of the paper provided.
Your summary should extract the most crucial points that can help researchers quickly understand the most vital information of the paper.
Create a bullet-point summary that outlines the method. Follow this up with a concise paragraph that encapsulates the main results.
The summary is designed for busy researchers that quickly need to grasp the newest trends in Large Language Models.
The tone should be professional and clear.
Text to summarize: MY TEXT TO SUMMARIZE<|end|>
<|endoftext|>
```

```

$ Generate the output
outputs = pipe(messages)
print(outputs[0]["generated_text"])

Key Findings:
- The Transformer model utilizes attention mechanisms to improve the speed and performance of deep learning models, particularly in neural machine translation tasks.
- It outperforms the Google Neural Machine Translation model in specific tasks and is recommended by Google Cloud for their Cloud TPU offering.
- The Transformer model consists of an encoding component and a decoding component, both composed of stacks of identical encoders and decoders.
- The encoders use self-attention layers to process input sequences, while the decoders incorporate attention layers to focus on relevant parts of the input.
- The model employs embedding algorithms to convert input words into vectors of size 512, which are then processed through the encoder's layers.
- The Transformer model's architecture allows for parallelization, enabling faster training and improved performance.

Summary:
- The Transformer model, introduced in the paper "Attention is All You Need," revolutionizes deep learning models by leveraging attention mechanisms to enhance performance and training speed.
- It surpasses the Google Neural Machine Translation model in specific tasks and is recommended by Google Cloud for their Cloud TPU offering.
- The model comprises an encoding component and a decoding component, each consisting of stacks of identical encoders and decoders.
- Encoders utilize self-attention layers to process input sequences, while decoders incorporate attention layers to focus on relevant input parts.
- Embedding algorithms convert input words into vectors of size 512, which are processed through the encoder's layers.
- The Transformer model's architecture enables parallelization, resulting in faster training and improved performance.

The Transformer model, introduced in the paper "Attention is All You Need," revolutionizes deep learning models by leveraging attention mechanisms to enhance performance and training speed. It

```

如果把 data_format 从 query 中去除，得到的结果如下：

```

# The full prompt - remove and add pieces to view its impact on the generated output
query = persona + instruction + context + audience + tone + data

The following generation flags are not valid and may be ignored: ['temperature']. Set 'TRANSFORMERS_VERBOSITY:info' for more details.
The Transformer model, introduced in the paper "Attention is All You Need," represents a significant advancement in the field of deep learning, particularly for tasks like neural machine translation. It leverages the attention

```

① In-Context Learning 上下文学习

分为三种：zero-shot prompt 零样本提示、one-shot prompt 单样本提示和 few-shot prompt 少量样本提示

在向 LLM 提问时，提供少量的示例是非常有效的。

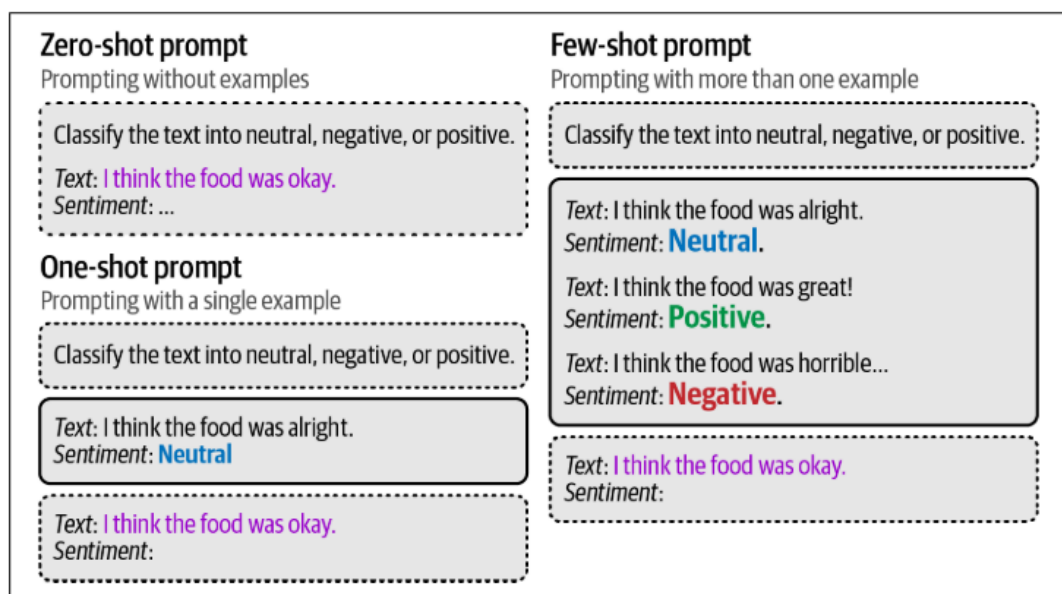


Figure 6-13. An example of a complex prompt with many components.

② Chain Prompting 链式引导提示

将一个提示的输出用作下一个提示的输入，从而创建一个解决问题的连续交互链。虽然我们可以要求 LLM 一次性完成任务，但我们可以将问题分解成多个部分。如图 6-14 所示，我们得到了一个顺序管道，它首先创建产品名，使用产品名和

产品特性作为输入来创建口号，最后，使用特性、产品名和口号来创建销售推销。这种链式引导提示允许 LLM 花费更多的时间在每一个单独的问题上，而不是一次性解决所有问题。

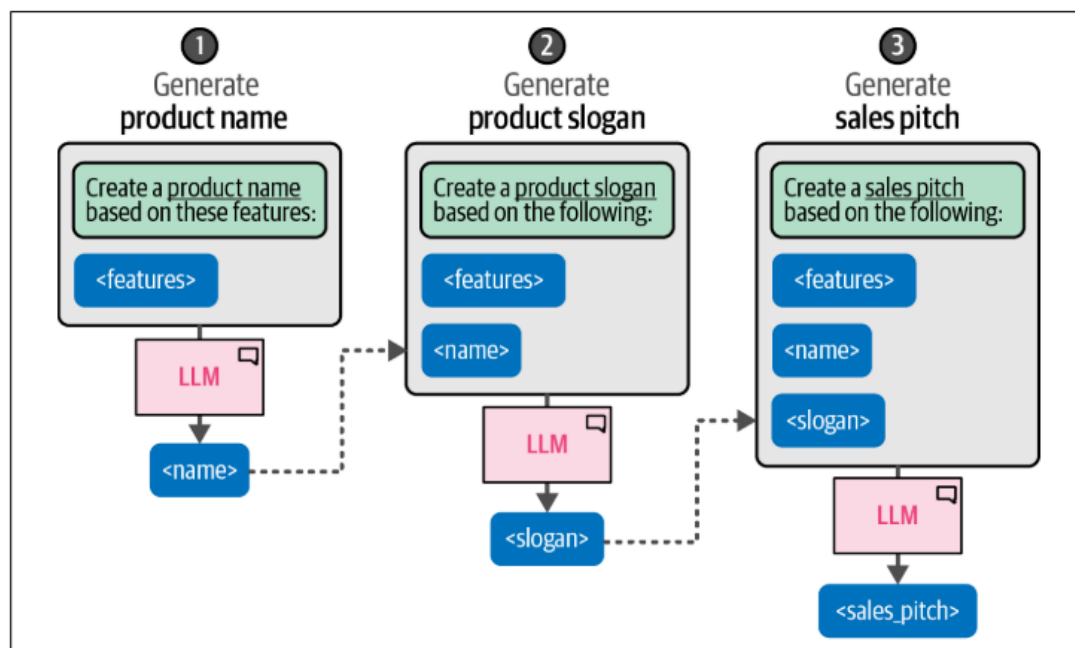


Figure 6-14. Using a description of a product's features, chain prompts to create a suitable name, slogan, and sales pitch.

三. 生成推理 Reasoning

赋予模型模仿自我反思形式的能力。一种有意识的、缓慢的、合乎逻辑的思维过程，类似于头脑风暴和自我反省。模仿人类推理的思维过程。

① Chain-of-Thought: Think Before Answering 思维链：先思考再回答

在提示中提供推理示例，演示了模型在生成响应之前应该做的推理。这些推理过程被称为“思考”。这对于复杂程度较高的任务，比如数学问题，有极大的帮助。添加这个推理步骤允许模型在推理过程中分配更多的计算，而不是直接生成答案。

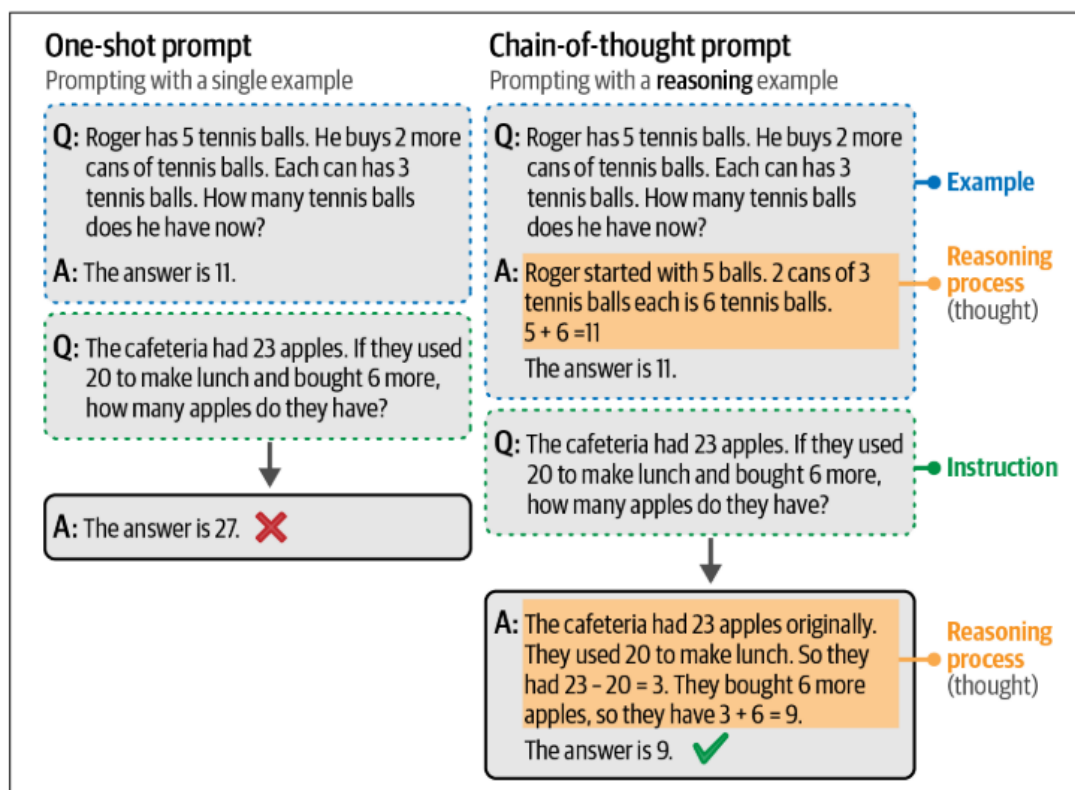


Figure 6-15. Chain-of-thought prompting uses reasoning examples to persuade the generative model to use reasoning in its answer.

尽管思维链是增强生成模型输出的好方法，但它需要提示中提供一个或多个推理示例，而用户可能无法提供这些示例。我们可以简单地要求模型提供推理过程（zero-shot chain-of-thought）。一个常见且有效的方法是使用短语“Let's think step-by-step”。

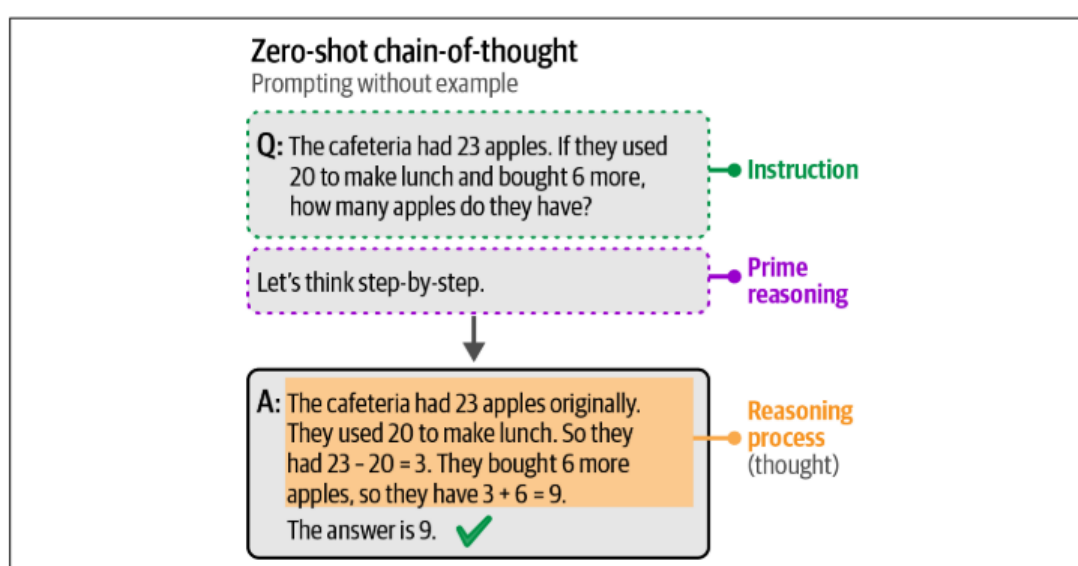


Figure 6-16. Chain-of-thought prompting without using examples. Instead, it uses the phrase “Let's think step-by-step” to prime reasoning in its answer.

② Self-Consistency 自一致性，采样输出

如果我们通过参数 `temperature` 和 `top_p` 等参数允许一定程度的创造性，那么多次使用相同的提示可能会导致不同的结果。因此，输出的质量可能会根据令牌的随机选择而提高或降低。向生成模型多次询问相同的提示，并将多数结果作为最终答案。在这个过程中，还可以加入思维链提示，提高输出的质量。

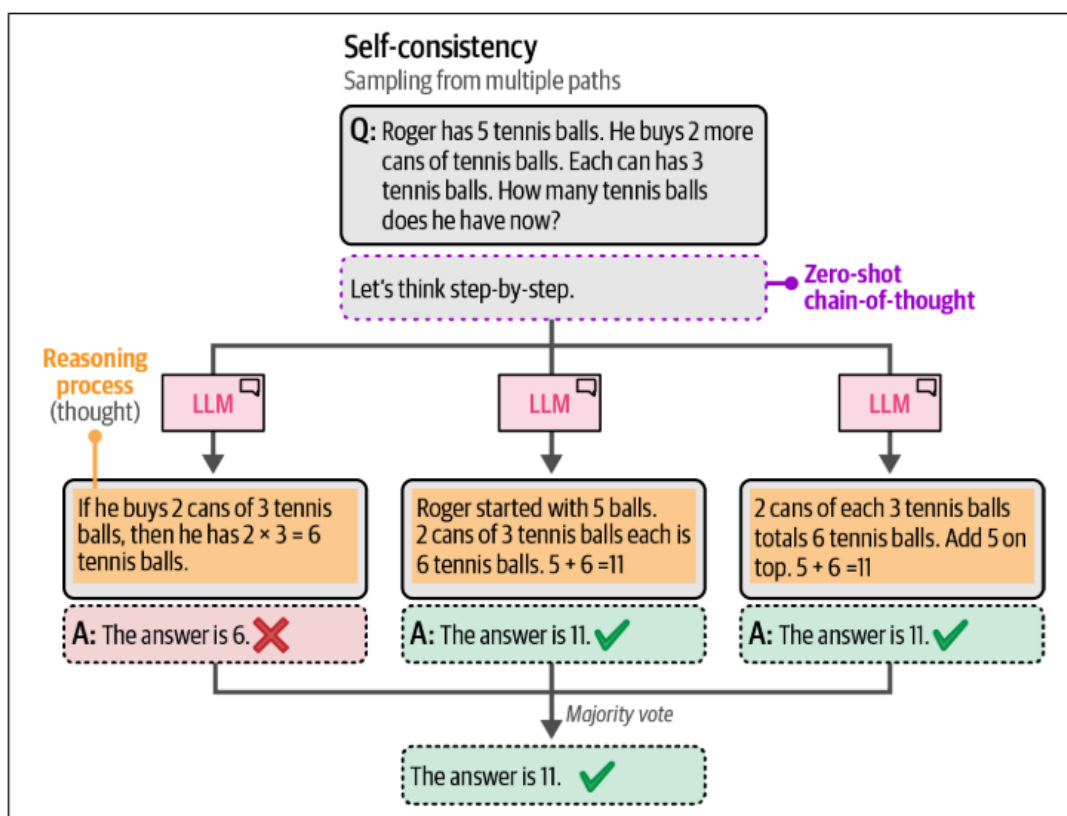


Figure 6-17. By sampling from multiple reasoning paths, we can use majority voting to extract the most likely answer.

③ Tree-of-Thought: Exploring Intermediate Steps 思维树：探索推理过程

当面对一个需要多个推理步骤的问题时，通常将其分解成几个部分。在每一步时，如图 6-18 所示，生成模型被提示探索问题的不同解决方法，然后它投票选出最佳解决方案，并进行下一步。但这种方法需要多次调用生成模型，这大大降低了应用程序的速度。

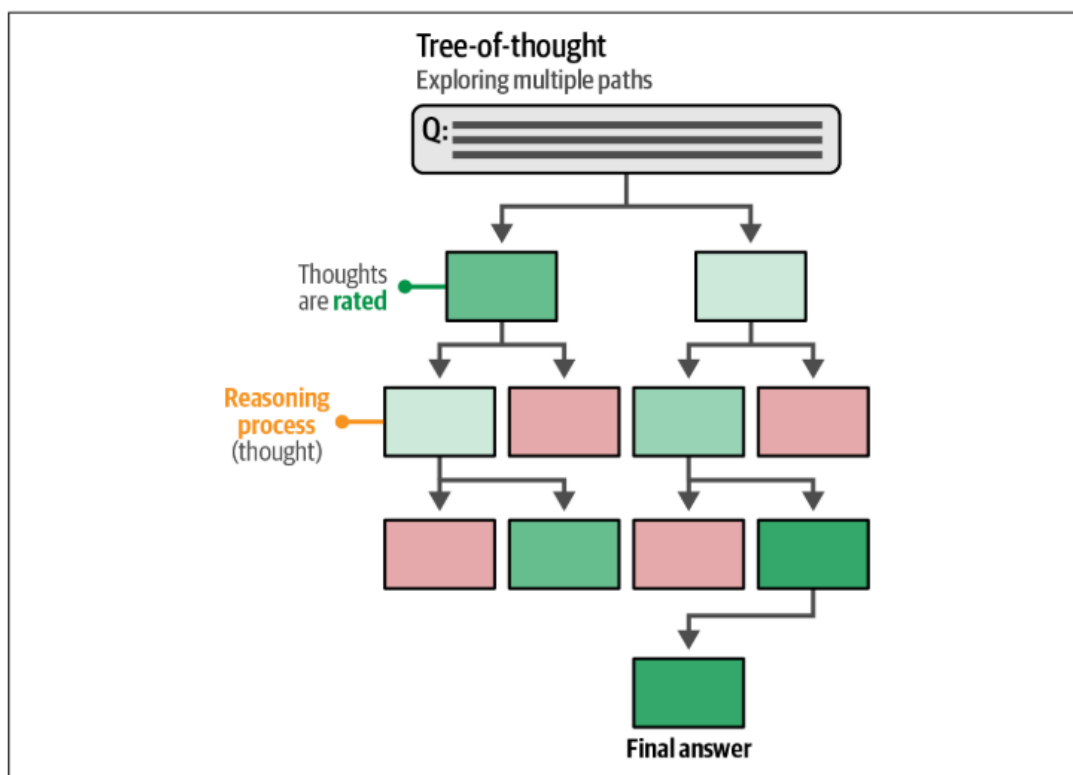


Figure 6-18. By leveraging a tree-based structure, generative models can generate intermediate thoughts to be rated. The most promising thoughts are kept and the lowest are pruned.

改进的方法是将思维树框架转化为一种简单的提示技巧，并非多次调用生成模型，而是要求模型通过模拟多个专家之间的对话，来模仿推理过程。这些专家将互相质疑，直到他们达成共识。

```
# Zero-shot tree-of-thought
zeroshot_tot_prompt = [
    {"role": "user", "content": "Imagine three different experts are answering this question. All experts will write down 1 step of their thinking, then share it with the group. Then all experts will go on to the next step, etc. If any expert realizes they're wrong at any point then they leave. The question is 'The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?' Make sure to discuss the results."}
]
```

想象一下，三个不同的专家正在回答这个问题。所有的专家都会写下他们思考的一个步骤，然后与团队分享。然后，所有的专家将继续进行下一步，等等。如果任何专家在任何时候意识到他们错了，那么他们就会离开。问题是“自助餐厅有 23 个苹果。”如果他们用了 20 个苹果做午餐，又买了 6 个，他们有多少个苹果？”一定要讨论结果。


```
# Generate the output
outputs = pipe(zeroshot_tot_prompt)
print(outputs[0]["generated_text"])

Expert 1: Step 1 - Start with the initial number of apples: 23 apples.

Expert 2: Step 1 - Subtract the apples used for lunch:  $23 - 20 = 3$  apples remaining.

Expert 3: Step 1 - Add the newly bought apples:  $3 + 6 = 9$  apples.

Expert 1: Step 2 - Confirm the final count: The cafeteria has 9 apples.

Expert 2: Step 2 - Review the calculations:  $23 - 20 = 3$ , then  $3 + 6 = 9$ . The calculations are correct.

Expert 3: Step 2 - Agree with the result: The cafeteria indeed has 9 apples.

All experts agree on the final count: The cafeteria has 9 apples.
```

四. 输出检验 Output Verification

检验输出的原因包括：

- ① 结构；②是否有效；③伦理；④是否真实准确

Structured output

By default, most generative models create free-form text without adhering to specific structures other than those defined by natural language. Some use cases require their output to be structured in certain formats, like JSON.

默认情况下，大多数生成模型创建自由格式的文本，不遵循由自然语言定义的结构。一些用例要求它们的输出以某些格式结构化，比如 JSON。

Valid output

Even if we allow the model to generate structured output, it still has the capability to freely generate its content. For instance, when a model is asked to output either one of two choices, it should not come up with a third.

即使允许模型生成结构化的输出，但它仍然具有自由生成内容的能力。例如，当一个模型被要求输出两个选择中的任何一个时，它不应该给出第三个。

Ethics

Some open source generative models have no guardrails and will generate outputs that do not consider safety or ethical considerations. For instance, use cases might require the output to be free of profanity, personally identifiable information (PII), bias, cultural stereotypes, etc.

一些开源生成模型没有 guardrails，会生成不考虑安全或道德因素的内容。例如，要求输出不含脏话、个人身份信息、偏见、文化刻板印象等。

Accuracy

Many use cases require the output to adhere to certain standards or performance. The aim is to double-check whether the generated information is factually accurate, coherent, or free from hallucination.

许多使用场景要求输出符合某些标准或性能。目的是反复检查生成的信息是否真实准确、连贯或没有幻觉。

有三种方法可以控制生成模型的输出：

- ① Examples 提供示例；②Grammar 约束；③Fine-tuning 微调（在 chapter12 中详解）
- ① Few-shot learning 是一种指导生成模型输出的有用技术。这种方法也可以推广到指导输出的结构。例如，我们希望生成模型为 RPG 游戏创建角色配置文件 JSON。

```
[ ] # Zero-shot learning: Providing no examples
zeroshot_prompt = [
    {"role": "user", "content": "Create a character profile for an RPG game in JSON format."}
]

# Generate the output
outputs = pipe(zeroshot_prompt)
print(outputs[0]["generated_text"])
```

```
```json
{
 "characterProfile": {
 "name": "Eldrin Stormbringer",
 "class": "Warlock",
 "race": "Half-Elf",
 "age": 27,
 "gender": "Male",
 "alignment": "Chaotic Good",
 "background": "Rogue",
 ...
 },
 "attributes": {
 "strength": 10,
 "dexterity": 17,
 "constitution": 12,
 "intelligence": 12,
 "wisdom": 10,
 "charisma"
```

但模型的输出并不完全符合 JSON 格式，不能直接使用。

```

One-shot learning: Providing an example of the output structure
one_shot_template = """Create a short character profile for an RPG game. Make sure to only use this format:

{
 "description": "A SHORT DESCRIPTION",
 "name": "THE CHARACTER'S NAME",
 "armor": "ONE PIECE OF ARMOR",
 "weapon": "ONE OR MORE WEAPONS"
}
"""

one_shot_prompt = [
 {"role": "user", "content": one_shot_template}
]

Generate the output
outputs = pipe(one_shot_prompt)
print(outputs[0]["generated_text"])

```

我们可以给模型提供预期格式的示例。

```

{
 "description": "A cunning rogue with a mysterious past, skilled in stealth and deception.",
 "name": "Lysandra Shadowstep",
 "armor": "Leather Cloak of the Night",
 "weapon": "Dagger of Whispers, Throwing Knives"
}

```

## ② Grammar: Constrained Sampling

Few-shot learning 有一个很大的缺点，即不能明确地阻止某些输出的生成。虽然我们引导模型并给它示例，但它可能仍然不会完全遵循它。目前，有一些包被开发出来以约束和检验生成模型的输出：如 Guidance、guardrails 和 LMQL（github 上可以找到）。在某种程度上，它们利用生成模型来验证自己的输出，如图 6-19 所示。生成模型将输出作为新的提示检索，并尝试基于一些预定义的 guardrails 来验证它。

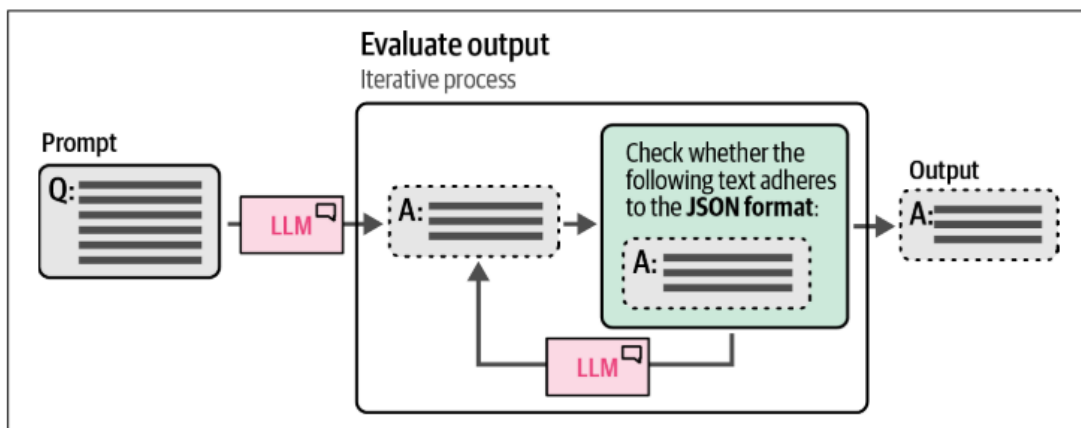


Figure 6-19. Use an LLM to check whether the output correctly follows our rules.

这个过程还可以用来控制输出的格式：

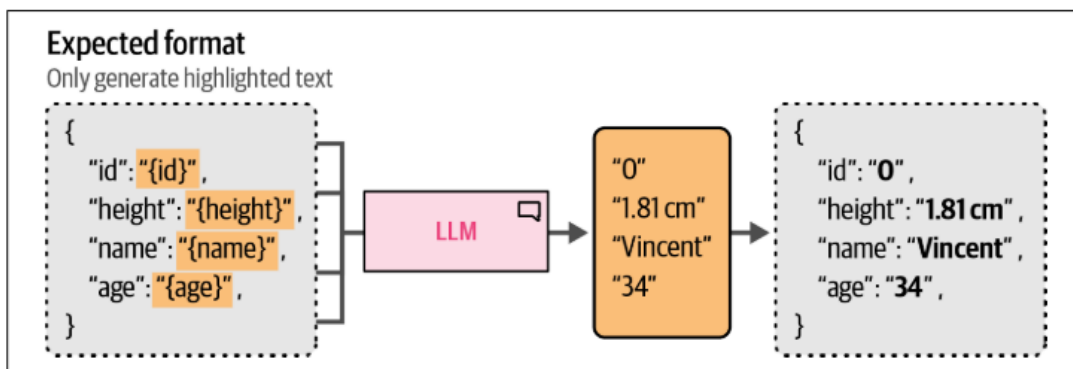


Figure 6-20. Use an LLM to generate only the pieces of information we do not know beforehand.

还可以再 token 的采样过程中执行检验，而不是输出完毕后再对输出进行检验。在对 token 进行采样时，我们可以定义一些语法或规则，LLM 在选择下一个 token 时应该遵循这些语法或规则。例如，如果我们要求模型在执行情感分类时返回“positive”、“negative”或“neutral”，它可能仍然会返回其他内容。如图 6-21 所示，通过约束采样过程，我们可以让 LLM 只输出我们感兴趣的内容。

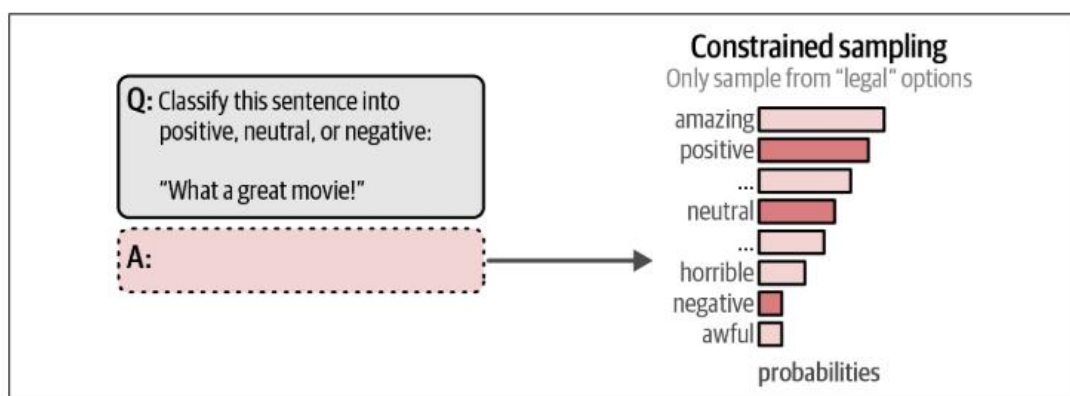


Figure 6-21. Constrain the token selection to only three possible tokens: “positive,” “neutral,” and “negative.”

在 token 采样过程中执行检验，示例：

```
from llama_cpp.llama import Llama

Load Phi-3
llm = Llama.from_pretrained(
 repo_id="microsoft/Phi-3-mini-4k-instruct-gguf",
 filename="*fp16.gguf",
 n_gpu_layers=-1,
 n_ctx=2048,
 verbose=False
)
```

用 llama\_cpp-python 加载压缩后的模型。使用内部 JSON 语法生成输出。将 response\_format 指定为 JSON 对象。在底层逻辑上，它将应用 JSON 语法来确保输出符合该格式。

```
[] # Generate output
output = llm.create_chat_completion(
 messages=[
 {"role": "user", "content": "Create a warrior for an RPG in JSON format."},
],
 response_format={"type": "json_object"},
 temperature=0,
)['choices'][0]['message']['content']

import json

Format as json
json_output = json.dumps(json.loads(output), indent=4)
print(json_output)
```